커밋 스킴과 머클 트리 분석

개요

이 문서는 Plonky3에서 구현된 다항식 커밋 스킴(Polynomial Commitment Scheme)과 벡터 커밋 스킴(Vector Commitment Scheme)을 분석합니다. 특히 일반화된 머클 트리 구현과 FRI 기반 다항식 커밋 스킴의 동작 원리를 살펴보고, 다양한 해시 함수(Poseidon, Keccak, BLAKE3)의 특성과 보안 모델을 비교 분석합니다.

커밋 스킴은 영지식 증명 시스템의 핵심 구성 요소로, 증명자가 특정 데이터에 대해 커밋한 후 나중에 그 데이터의 일부를 공개할 수 있게 해주는 암호학적 프리미티브입니다. Plonky3는 이러한 커밋 스킴을 모듈화된 방식으로 구현하여 다양한 암호학적 요구사항을 만족시킵니다.

커밋 스킴의 기본 개념

커밋 스킴의 정의

커밋 스킴은 두 단계로 구성됩니다:

1. **커밋 단계 (Commit Phase)**: 데이터 `m` 에 대해 커밋 `c = Commit(m, r)` 을 생성 (여기서 `r` 은 랜덤성)
2. **오픈 단계 (Open Phase)**: 원본 데이터 `m` 과 랜덤성 `r` 을 공개하여 커밋의 유효성을 검증

커밋 스킴이 만족해야 하는 보안 속성:

- **Binding**: 커밋 후에는 다른 값으로 오픈할 수 없음
- **Hiding**: 커밋만으로는 원본 데이터에 대한 정보를 얻을 수 없음 (선택적)

다항식 커밋 스킴 (PCS)

다항식 커밋 스킴은 다항식 `f(x)` 에 커밋하고, 나중에 특정 점 `z` 에서의 값 `f(z)` 를 증명할 수 있게 해주는 특수한 커밋 스킴입니다.

```
pub trait Pcs<Challenge, Challenger> {
type Domain: PolynomialSpace;
type Commitment: Clone + Serialize + DeserializeOwned;
type ProverData;
type Proof: Clone + Serialize + DeserializeOwned;
// 다항식 평가값들에 커밋
fn commit(&self, evaluations: impl IntoIterator<Item = (Self::Domain, RowMajorM
-> (Self::Commitment, Self::ProverData);
// 특정 점들에서 다항식 오픈
fn open(&self, commitment_data_with_opening_points: Vec<(&Self::ProverData, Vec
challenger: &mut Challenger) -> (OpenedValues<Challenge>, Self::Proof);
// 오픈 증명 검증
fn verify(&self, commitments_with_opening_points: Vec<(Self::Commitment, Vec<(S
proof: &Self::Proof, challenger: &mut Challenger) -> Result<(), Self::Error>;
}
```

Mixed Matrix Commitment Scheme (MMCS)

MMCS의 개념

MMCS는 벡터 커밋 스킴의 일반화로, 서로 다른 크기의 행렬들에 동시에 커밋하고 특정 행들을 오픈할 수 있게 해줍니다.

```
pub trait Mmcs<T: Send + Sync + Clone>: Clone {
```

```
type ProverData<M>;

type Commitment: Clone + Serialize + DeserializeOwned;

type Proof: Clone + Serialize + DeserializeOwned;

// 행렬 배치에 커밋

fn  commit<M: Matrix<T>>(&self, inputs: Vec<M>) -> (Self::Commitment, Self::Prov

// 특정 행 인덱스에서 배치 오픈

fn  open_batch<M: Matrix<T>>(&self, index: usize, prover_data: &Self::ProverData

-> BatchOpening<T, Self>;

// 배치 오픈 검증

fn  verify_batch(&self, commit: &Self::Commitment, dimensions: &[Dimensions],

index: usize, batch_opening: BatchOpeningRef<T, Self>) -> Result<(), Self::Error

}
```

배치 오픈 의미론

MMCS의 핵심 특징은 서로 다른 높이의 행렬들을 하나의 트리에 통합하는 방식입니다:

- 가장 큰 높이를 가진 행렬이 기준이 됨
- 작은 행렬들은 해당하는 레벨에서 비트 시프트를 통해 인덱스 조정
- 인덱스 `j = index >> (log2_ceil(max_height) - log2_ceil(matrix_height))`

일반화된 머클 트리 구현

MerkleTree 구조

Plonky3의 머클 트리는 서로 다른 높이의 행렬들을 지원하는 일반화된 구현입니다:

```
pub struct MerkleTree<F, W, M, const DIGEST_ELEMS: usize> {
// 모든 리프 행렬들 (삽입 순서대로)
pub(crate) leaves: Vec<M>,
// 중간 다이제스트 레이어들
// 인덱스 0이 리프 바로 위 레이어, 마지막이 루트
pub(crate) digest_layers: Vec<Vec<[W; DIGEST_ELEMS]>>,
_phantom: PhantomData<F>,
}
```

트리 구성 알고리즘

1. **높이별 정렬**: 행렬들을 높이 순으로 정렬
2. **첫 번째 다이제스트 레이어**: 가장 큰 행렬들의 모든 행을 해시
3. **레이어별 압축**: 각 레이어에서 2:1 압축과 작은 행렬 주입을 반복

```
fn first_digest_layer<P, PW, H, M, const DIGEST_ELEMS: usize>(
h: &H, tallest_matrices: Vec<&M>
) -> Vec<[PW::Value; DIGEST_ELEMS]> {
let max_height = tallest_matrices[0].height();
let max_height_padded = if max_height == 1 { 1 } else { max_height + max_heigh
let mut digests = vec![default_digest; max_height_padded];
// SIMD 병렬 해싱
digests[0..max_height].par_chunks_exact_mut(width).enumerate().for_each(|(i, dig
```

```
let first_row = i * width;
let packed_digest: [PW; DIGEST_ELEMS] = h.hash_iter(
tallest_matrices.iter().flat_map(|m| m.vertically_packed_row(first_row))
);
for (dst, src) in digests_chunk.iter_mut().zip(unpack_array(packed_digest)) {
*dst = src;
}
});
digests
}
```

압축과 주입 과정

```
fn compress_and_inject<P, PW, H, C, M, const DIGEST_ELEMS: usize>(
prev_layer: &[[PW::Value; DIGEST_ELEMS]],
matrices_to_inject: Vec<&M>,
h: &H, c: &C
) -> Vec<[PW::Value; DIGEST_ELEMS]> {
// 이전 레이어 압축
let compressed = compress_pairs(prev_layer, c);
// 새로운 행렬 행들 해시
let injected_hashes = hash_matrix_rows(matrices_to_inject, h);
```

```
// 압축된 값과 주입된 해시 결합

combine_compressed_and_injected(compressed, injected_hashes, c)

}
```

MerkleTreeMmcs 구현

구조와 특징

```
pub struct MerkleTreeMmcs<P, PW, H, C, const DIGEST_ELEMS: usize> {

hash: H, // 리프 해시 함수

compress: C, // 내부 노드 압축 함수

_phantom: PhantomData<(P, PW)>,

}
```

커밋 과정

```
impl<P, PW, H, C, const DIGEST_ELEMS: usize> Mmcs<P::Value> for MerkleTreeMmcs<P

fn commit<M: Matrix<P::Value>>(&self, inputs: Vec<M>) -> (Self::Commitment, Sel

let tree = MerkleTree::new::<P, PW, H, C>(&self.hash, &self.compress, inputs);

let root = tree.root();

(root, tree)

}

}
```

배치 오픈 과정

```
fn open_batch<M: Matrix<P::Value>>(&self, index: usize, prover_data: &MerkleTre
-> BatchOpening<P::Value, Self> {
let max_height = self.get_max_height(prover_data);
let log_max_height = log2_ceil_usize(max_height);
// 각 행렬에서 해당하는 행 추출
let openings = prover_data.leaves.iter().map(|matrix| {
let log2_height = log2_ceil_usize(matrix.height());
let bits_reduced = log_max_height - log2_height;
let reduced_index = index >> bits_reduced;
matrix.row(reduced_index).unwrap().into_iter().collect()
}).collect_vec();
// 머클 경로 생성
let proof = (0..log_max_height)
.map(|i| prover_data.digest_layers[i][(index >> i) ^ 1])
.collect();
BatchOpening::new(openings, proof)
}
```

검증 과정

검증은 리프에서 루트까지의 경로를 재구성하여 커밋된 루트와 비교합니다:

```
fn verify_batch(&self, commit: &Self::Commitment, dimensions: &[Dimensions],
mut index: usize, batch_proof: BatchOpeningRef<P::Value, Self>) -> Result<(), S
let (opened_values, opening_proof) = batch_proof.unpack();
// 현재 높이에서 행렬 오픈값들 해시
let mut root = self.hash.hash_iter_slices(/* 현재 높이의 행렬들 */);
for &sibling in opening_proof {
// 형제 노드와 결합
let (left, right) = if index & 1 == 0 { (root, sibling) } else { (sibling, root
root = self.compress.compress([left, right]);
index >>= 1;
// 새로운 행렬 행들이 있다면 주입
if let Some(next_height_openings) = /* 다음 높이 행렬들 */ {
let next_digest = self.hash.hash_iter_slices(next_height_openings);
root = self.compress.compress([root, next_digest]);
}
}
// 계산된 루트와 커밋 비교
if commit == &root { Ok(()) } else { Err(RootMismatch) }
}
```

FRI 기반 다항식 커밋 스킴

TwoAdicFriPcs 구조

FRI(Fast Reed-Solomon Interactive Oracle Proof)를 기반으로 한 다항식 커밋 스킴:

```
pub struct TwoAdicFriPcs<Val, Dft, InputMmcs, FriMmcs> {
pub(crate) dft: Dft, // DFT 구현
pub(crate) mmcs: InputMmcs, // 입력 MMCS
pub(crate) fri: FriParameters<FriMmcs>, // FRI 매개변수
_phantom: PhantomData<Val>,
}
```

커밋 과정

1. **LDE 계산**: 다항식 평가값들을 Low Degree Extension으로 확장
2. **비트 역순 정렬**: FRI 프로토콜 요구사항에 맞게 정렬
3. **MMCS 커밋**: 확장된 평가값들에 커밋

```
fn commit(&self, evaluations: impl IntoIterator<Item = (Self::Domain, RowMajorM
-> (Self::Commitment, Self::ProverData) {
let ldes: Vec<_> = evaluations.into_iter().map(|(domain, evals)| {
assert_eq!(domain.size(), evals.height());
// 코셋 LDE 계산
let shift = Val::GENERATOR / domain.shift();
self.dft.coset_lde_batch(evals, self.fri.log_blowup, shift)
.bit_reverse_rows()
```

```
.to_row_major_matrix()

}).collect();

// 비트 역순 LDE들에 커밋

self.mmcs.commit(ldes)

}
```

오픈 과정

FRI 기반 오픈은 복잡한 최적화를 포함합니다:

1. **배치 결합**: 여러 다항식과 오픈 포인트를 `alpha` 챌린지로 결합
2. **몫 다항식 계산**: `(f(x) - f(z))/(x - z)` 형태의 몫 다항식들 계산
3. **FRI 증명**: 몫 다항식들이 저차수임을 FRI로 증명

```
fn open(&self, commitment_data_with_opening_points: Vec<ProverDataWithOpeningPo

challenger: &mut Challenger) -> (OpenedValues<Challenge>, Self::Proof) {

// 1. 베리센트릭 보간으로 오픈 값들 계산

let all_opened_values = compute_opened_values_with_barycentric_interpolation(/*

// 2. 배치 결합 챌린지 샘플링

let alpha: Challenge = challenger.sample_algebra_element();

let alpha_powers = compute_alpha_powers(alpha, global_max_width);

// 3. 몫 다항식들 계산 및 결합

let mut reduced_openings = compute_quotient_polynomials(

&mats_and_points, &all_opened_values, &alpha_powers, &inv_denoms

);
```

```
// 4. FRI 증명 생성
let fri_input = reduced_openings.into_iter().rev().flatten().collect_vec();
let fri_proof = prover::prove_fri(&folding, &self.fri, fri_input, challenger, /
(all_opened_values, fri_proof)
}
```

역 분모 계산 최적화

오픈 과정에서 가장 중요한 최적화 중 하나는 역 분모의 배치 계산입니다:

```
fn compute_inverse_denominators<F: TwoAdicField, EF: ExtensionField<F>, M: Matr
mats_and_points: &[(Vec<M>, &Vec<Vec<EF>>)],
coset: &[F]
) -> LinearMap<EF, Vec<EF>> {
// 각 포인트 z에 대해 최대 높이 찾기
let mut max_log_height_for_point: LinearMap<EF, usize> = LinearMap::new();
for (mats, points) in mats_and_points {
for (mat, points_for_mat) in izip!(mats, *points) {
let log_height = log2_strict_usize(mat.height());
for &z in points_for_mat {
max_log_height_for_point.entry(z).and_modify(|lh| *lh = (*lh).max(log_height))
.or_insert(log_height);
}
}
```

```
}

// 각 포인트에 대해 역 분모 계산

max_log_height_for_point.into_iter().map(|(z, log_height)| {

(z, batch_multiplicative_inverse(

&coset[..(1 << log_height)].iter().map(|&x| z - x).collect_vec()

))

}).collect()

}
```

해시 함수 비교 분석

Poseidon

특징:

- 유한체 기반 해시 함수
- STARK 친화적 설계
- 낮은 곱셈 복잡도

구현:

```
pub  struct Poseidon<F, const WIDTH: usize, const ALPHA: u64> {

round_constants: Vec<[F; WIDTH]>,

mds_matrix: MdsMatrix<F, WIDTH>,

}
```

```
impl<F: Field, const WIDTH: usize, const ALPHA: u64> CryptographicHasher<F, [F;
for Poseidon<F, WIDTH, ALPHA> {
fn hash_iter<I>(&self, input: I) -> [F; WIDTH] where I: IntoIterator<Item = F>
let mut state = [F::ZERO; WIDTH];
// 입력 흡수
for chunk in input.into_iter().chunks(WIDTH - 1).into_iter() {
for (i, elem) in chunk.enumerate() {
state[i + 1] = elem;
}
self.permute(&mut state);
}
state
}
}
```

보안 모델:

- 스폰지 구조 기반
- 대수적 공격에 대한 저항성
- 유한체 크기에 따른 보안 수준

Keccak

특징:

- SHA-3 표준의 기반

- 비트 기반 연산
- 높은 보안성과 표준화

구현:

```
pub struct Keccak256Hash;

impl CryptographicHasher<u8, [u8; 32]> for Keccak256Hash {
fn hash_iter<I>(&self, input: I) -> [u8; 32] where I: IntoIterator<Item = u8> {
let mut hasher = Keccak::v256();
let input_bytes: Vec<u8> = input.into_iter().collect();
hasher.update(&input_bytes);
let mut output = [0u8; 32];
hasher.finalize(&mut output);
output
}
}
```

보안 모델:

- 스폰지 구조
- 차분 및 선형 암호분석에 대한 저항성
- NIST 표준화된 보안 수준

BLAKE3

특징:

- 고성능 해시 함수
- 병렬화 친화적
- ChaCha 기반 설계

구현:

```
pub struct Blake3;

impl CryptographicHasher<u8, [u8; 32]> for Blake3 {
fn hash_iter<I>(&self, input: I) -> [u8; 32] where I: IntoIterator<Item = u8> {
let input_bytes: Vec<u8> = input.into_iter().collect();
blake3::hash(&input_bytes).into()
}
}
```

보안 모델:

- Merkle-Damgård 구조의 개선
- ARX(Addition, Rotation, XOR) 연산 기반
- 높은 성능과 보안성의 균형

성능 비교

해시 함수	유한체 친화성	SIMD 최적화	병렬화	보안 수준
Poseidon	매우 높음	보통	보통	높음
Keccak	낮음	높음	낮음	매우 높음

| BLAKE3 | 낮음 | 매우 높음 | 매우 높음 | 높음 |

보안 모델 분석

Binding 속성

모든 커밋 스킴은 계산적 binding을 제공합니다:

- **머클 트리**: 해시 함수의 충돌 저항성에 의존
- **FRI PCS**: Reed-Solomon 코드의 거리 속성과 FRI soundness에 의존

Hiding 속성

Plonky3의 기본 구현은 non-hiding이지만, hiding 변형도 지원합니다:

```
pub trait Pcs<Challenge, Challenger> {
const ZK: bool; // Zero-knowledge 활성화 여부
fn get_opt_randomization_poly_commitment(&self, domain: Self::Domain)
-> Option<(Self::Commitment, Self::ProverData)> {
if Self::ZK {
// 랜덤화 다항식 생성
Some(self.commit_randomization_poly(domain))
} else {
None
}
}
}
```

Soundness 분석

FRI 기반 PCS의 soundness는 다음 요소들에 의존합니다:

1. **Blowup Factor**: `2^log_blowup`
2. **Query 수**: `num_queries`
3. **Proof of Work**: `proof_of_work_bits`

```
impl<M> FriParameters<M> {
pub const fn conjectured_soundness_bits(&self) -> usize {
self.log_blowup * self.num_queries + self.proof_of_work_bits
}
}
```

ethSTARK 추측에 따른 soundness:

- 각 쿼리당 `log_blowup` 비트의 soundness
- Proof of Work로 추가 보안 제공

실제 사용 예제

기본 머클 트리 사용

```
use p3_merkle_tree::MerkleTreeMmcs;
use p3_baby_bear::BabyBear;
use p3_poseidon::Poseidon;
```

```
type F = BabyBear;

type Hash = Poseidon<F, 16>;

type Compress = TruncatedPermutation<Poseidon<F, 16>, 2, 8, 16>;

type Mmcs = MerkleTreeMmcs<F, F, Hash, Compress, 8>;



// MMCS 초기화

let hash = Hash::new_from_rng(&mut rng);

let compress = Compress::new(hash.clone());

let mmcs = Mmcs::new(hash, compress);



// 행렬들에 커밋

let matrices = vec![

RowMajorMatrix::rand(&mut rng, 8, 4),

RowMajorMatrix::rand(&mut rng, 4, 6),

];

let (commitment, prover_data) = mmcs.commit(matrices);



// 특정 인덱스에서 오픈

let opening = mmcs.open_batch(3, &prover_data);



// 검증

let dimensions = vec![

Dimensions { height: 8, width: 4 },
```

```
Dimensions { height: 4, width: 6 },

];

mmcs.verify_batch(&commitment, &dimensions, 3, (&opening).into())?;
```

FRI PCS 사용

```
use p3_fri::TwoAdicFriPcs;


type Dft = Radix2DitParallel;

type Pcs = TwoAdicFriPcs<F, Dft, Mmcs, Mmcs>;


// PCS 초기화

let  dft = Dft::default();

let  fri_params = FriParameters {

log_blowup: 2,

log_final_poly_len: 0,

num_queries: 100,

proof_of_work_bits: 16,

mmcs: mmcs.clone(),

};

let  pcs = Pcs::new(dft, mmcs, fri_params);


// 다항식 평가값들에 커밋
```

```
let domain = pcs.natural_domain_for_degree(8);

let evaluations = vec![(domain, RowMajorMatrix::rand(&mut rng, 8, 3))];

let (commitment, prover_data) = pcs.commit(evaluations);



// 특정 점들에서 오픈

let opening_points = vec![vec![Challenge::from_base(F::from_u32(42))]];

let commitment_data = vec![(&prover_data, opening_points)];

let (opened_values, proof) = pcs.open(commitment_data, &mut challenger);



// 검증

let commitments_with_points = vec![(commitment, vec![(domain, vec![(

Challenge::from_base(F::from_u32(42)),

opened_values[0][0][0].clone()

)])])];

pcs.verify(commitments_with_points, &proof, &mut challenger)?;
```

성능 최적화 기법

SIMD 최적화

머클 트리 구성에서 SIMD 명령어를 활용한 병렬 해싱:

```
// 완전한 SIMD 배치들을 병렬로 처리

digests[0..max_height].par_chunks_exact_mut(width).enumerate().for_each(|(i, dig
```

```
let first_row = i * width;

let packed_digest: [PW; DIGEST_ELEMS] = h.hash_iter(

tallest_matrices.iter().flat_map(|m| m.vertically_packed_row(first_row))

);

for (dst, src) in digests_chunk.iter_mut().zip(unpack_array(packed_digest)) {

*dst = src;

}

});



// 남은 행들을 스칼라 방식으로 처리

for i in ((max_height / width) * width)..max_height {

unsafe {

digests[i] = h.hash_iter(tallest_matrices.iter().flat_map(|m| m.row_unchecked(i)

}

}
```

메모리 최적화

- **비트 역순 정렬**: 캐시 친화적인 메모리 접근 패턴
- **제자리 연산**: 불필요한 메모리 할당 최소화
- **배치 처리**: 여러 연산을 한 번에 처리하여 오버헤드 감소

확장 필드 최적화

FRI PCS에서 확장 필드 곱셈을 최소화하는 전략:

```
// 베이스 필드에서 가능한 한 많은 연산 수행
let  halve_inv_powers = g_inv.shifted_powers(F::ONE.halve()).collect_n(m.height(

// 확장 필드 곱셈은 마지막에 한 번만
m.par_rows().zip(halve_inv_powers).map(|(mut  row, halve_inv_power)| {
let (lo, hi) = row.next_tuple().unwrap();
(lo + hi).halve() + (lo - hi) * beta * halve_inv_power  // beta만 확장 필드
}).collect()
```

결론

Plonky3의 커밋 스킴 구현은 다음과 같은 특징을 가집니다:

1. **모듈화된 설계**: 다양한 해시 함수와 압축 함수를 플러그인 방식으로 지원
2. **일반화된 MMCS**: 서로 다른 크기의 행렬들을 효율적으로 처리
3. **최적화된 FRI PCS**: 복잡한 수학적 최적화를 통한 고성능 다항식 커밋
4. **다양한 보안 모델**: 각 해시 함수의 특성에 맞는 보안 분석

이러한 구현은 STARK 증명 시스템의 핵심 구성 요소로서 높은 성능과 보안성을 동시에 제공하며, 다양한 암호학적 요구사항을 만족시킬 수 있는 유연성을 갖추고 있습니다.